5. Markov Decision Process I

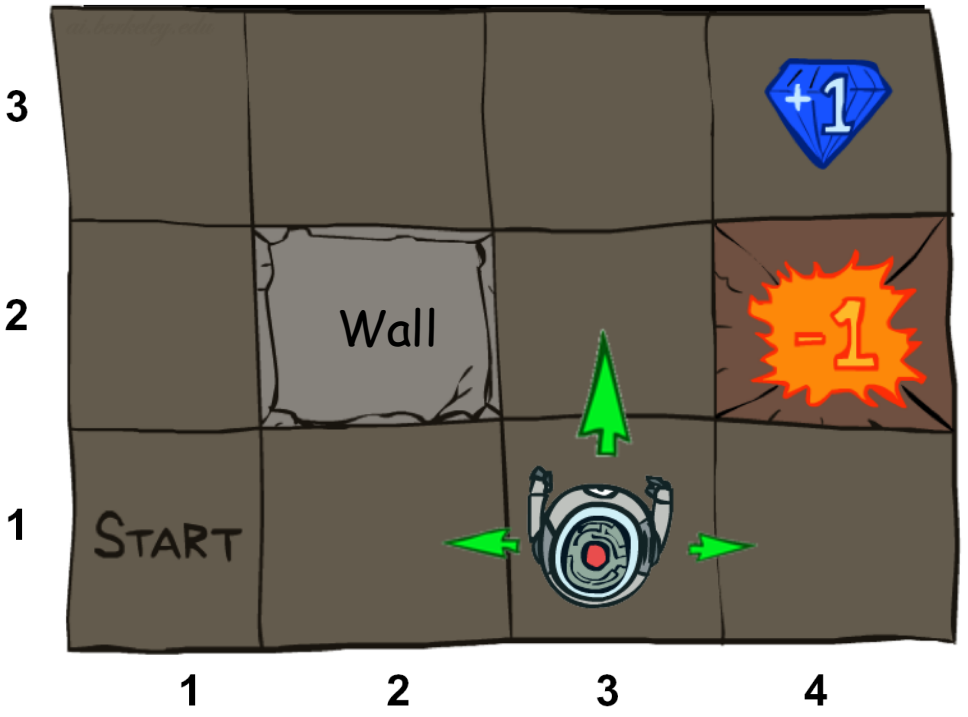
Ack: Berkeley AI course

Outline

- Non-Deterministic Search
- Markov Decision Process

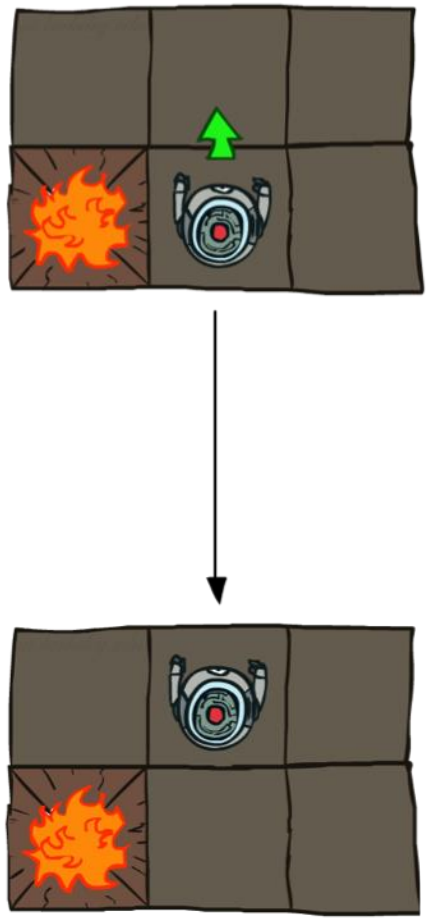
Example: Grid World

- A maze-like problem
  - The agent lives in a grid
  - Walls block the agent's path
- Noisy movement: actions do not always go as planned
  - 80% of the time, the action North takes the agent North (if there is no wall there)
  - 10% of the time, North takes the agent West; 10% East
  - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards
  - Small "living" reward each step (can be negative)
  - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards

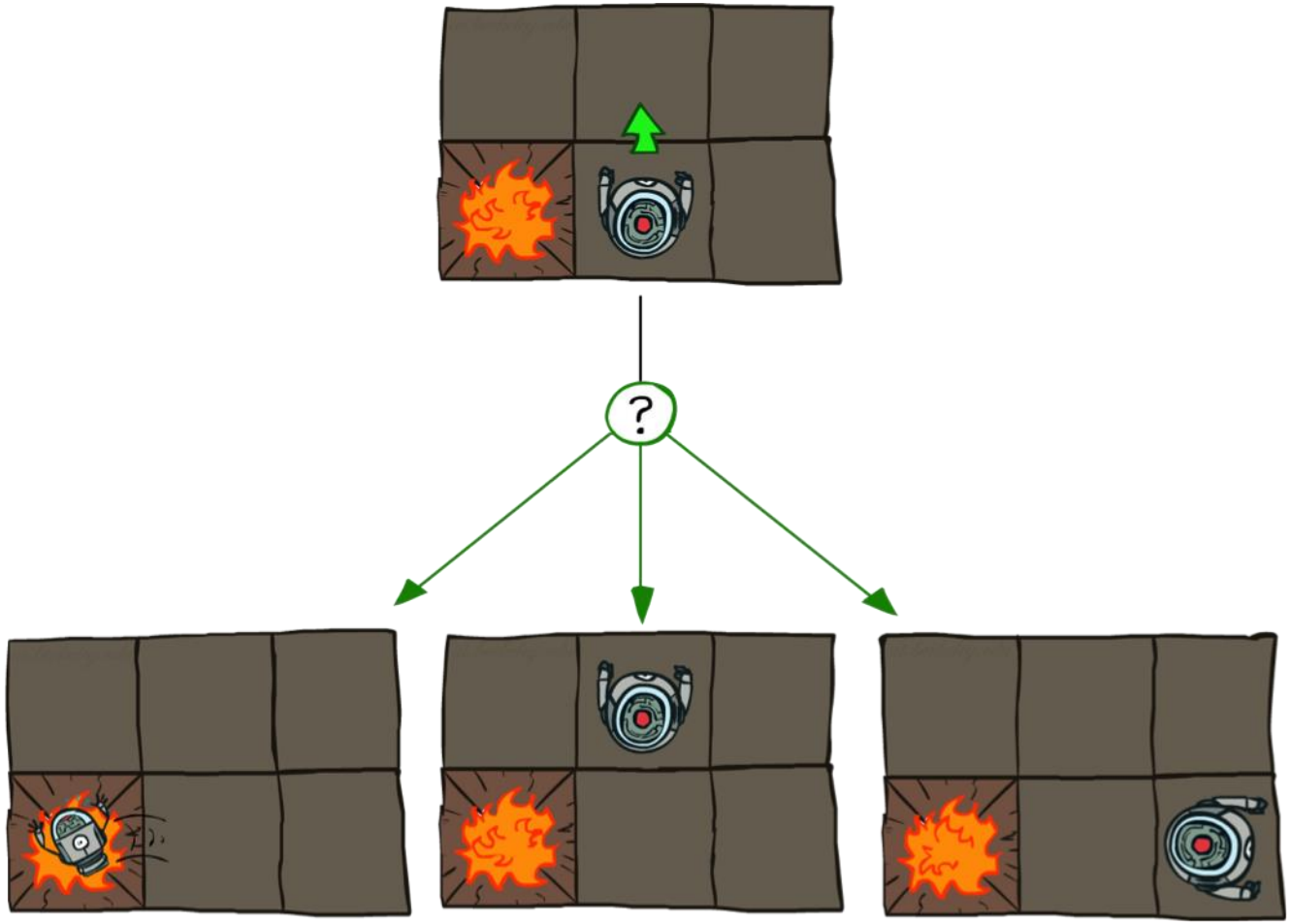


Grid World Actions

Deterministic Grid World



Stochastic Grid World

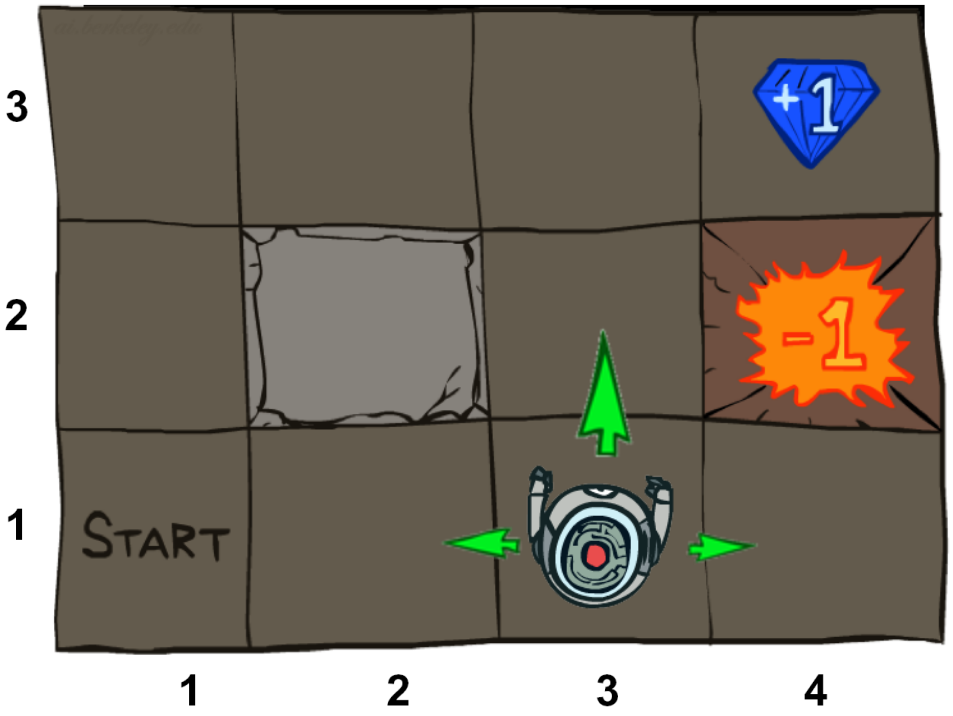


Outline

- Non-Deterministic Search
- **Markov Decision Process**

Markov Decision Processes

- An MDP is defined by:
  - A set of states $s \in S$
  - A set of actions $a \in A$
  - A transition function T(s, a, s')
    - Probability that a from s leads to s', i.e., P(s' | s, a)
    - Also called the model or the dynamics
  - A reward function R(s, a, s')
    - Sometimes just R(s) or R(s')
  - A start state
  - Maybe a terminal state



- MDPs are non-deterministic search problems

What is Markov about MDPs?

- "Markov" generally means that given the present state, the future and the past are independent

- For Markov decision processes, "Markov" means action outcomes depend only on the current state

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \ldots S_0 = s_0)$$

$$=$$

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

- This is just like search, where the successor function could only depend on the current state (not the history)

Andrey Markov
(1856-1922)

Policies

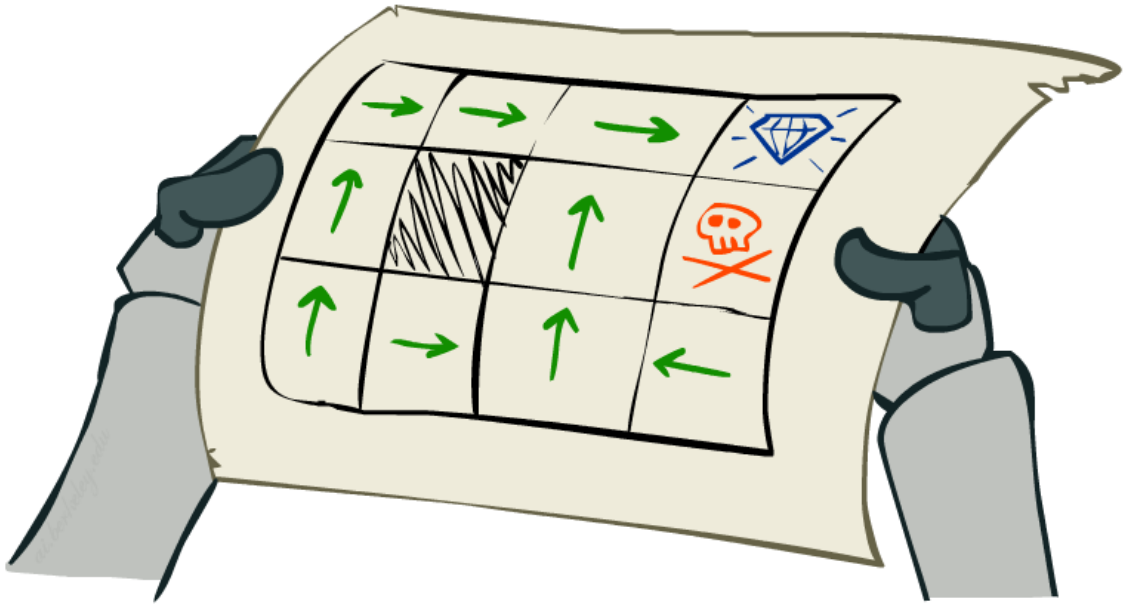
- In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal

- For MDPs, we want an optimal

  policy $\pi^*: S \rightarrow A$

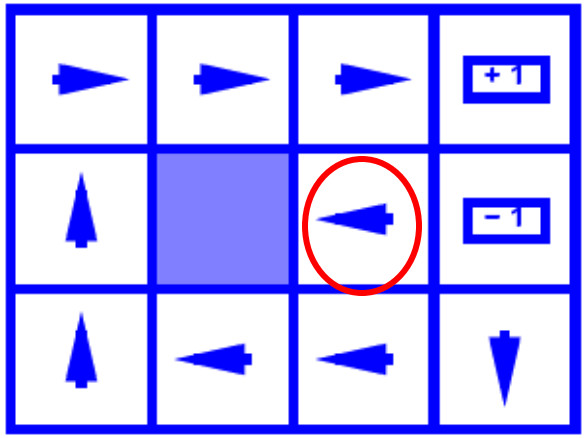
  - A policy π gives an action for each state
  - An optimal policy is one that maximizes expected utility if followed
  - An explicit policy defines a reflex agent

- Expectimax did not compute entire policies
  - It computes the action for a single state only

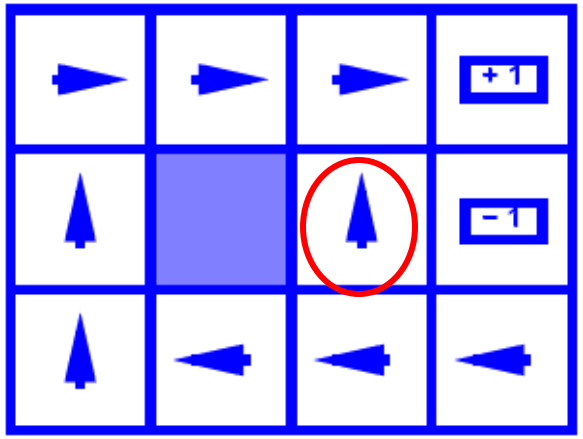


Optimal policy when R(s, a, s') = -0.03 for all non-terminals s

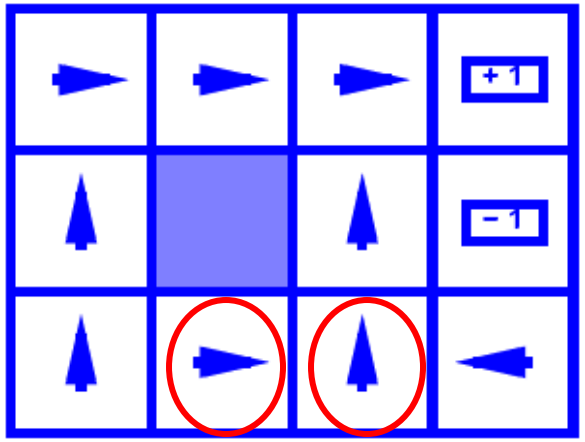
Optimal Policies



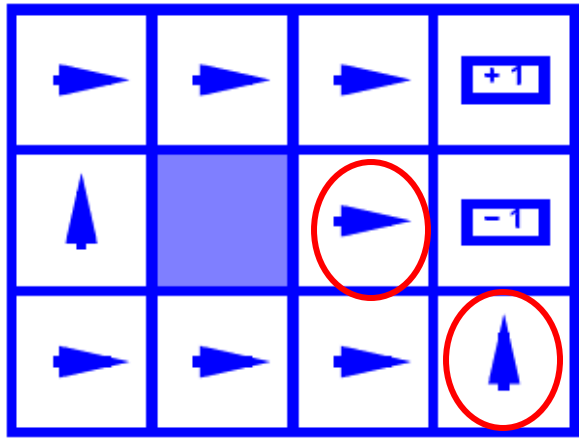
R(s) = -0.01



R(s) = -0.03



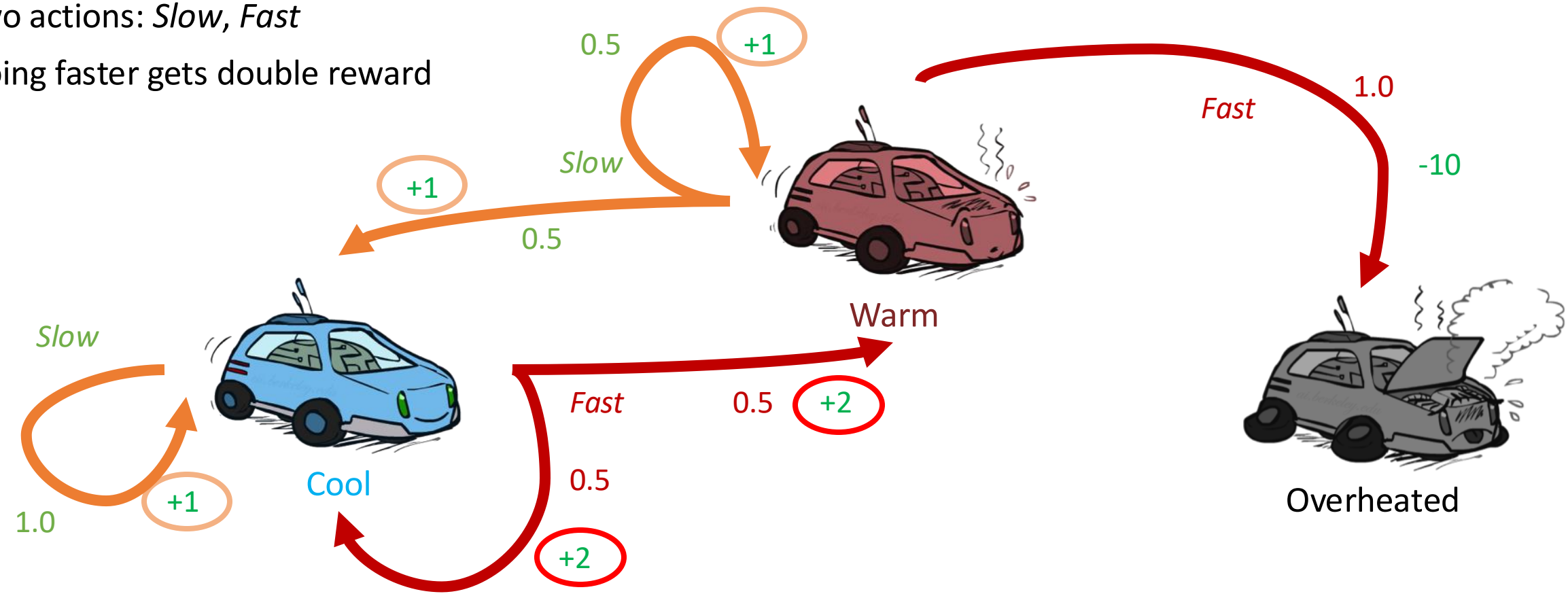
R(s) = -0.4



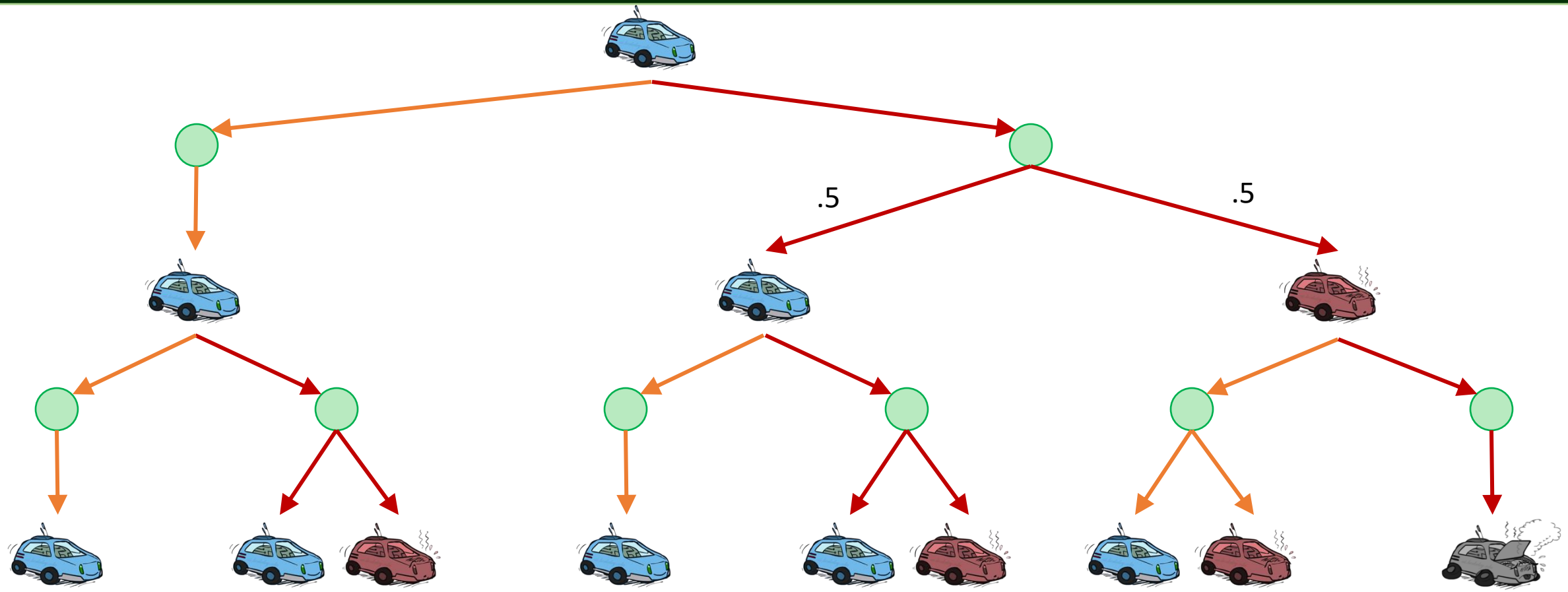
R(s) = -2.0

Example: Racing

- A robot car wants to travel far, quickly
- Three states: Cool, Warm, Overheated
- Two actions: *Slow*, *Fast*
- Going faster gets double reward

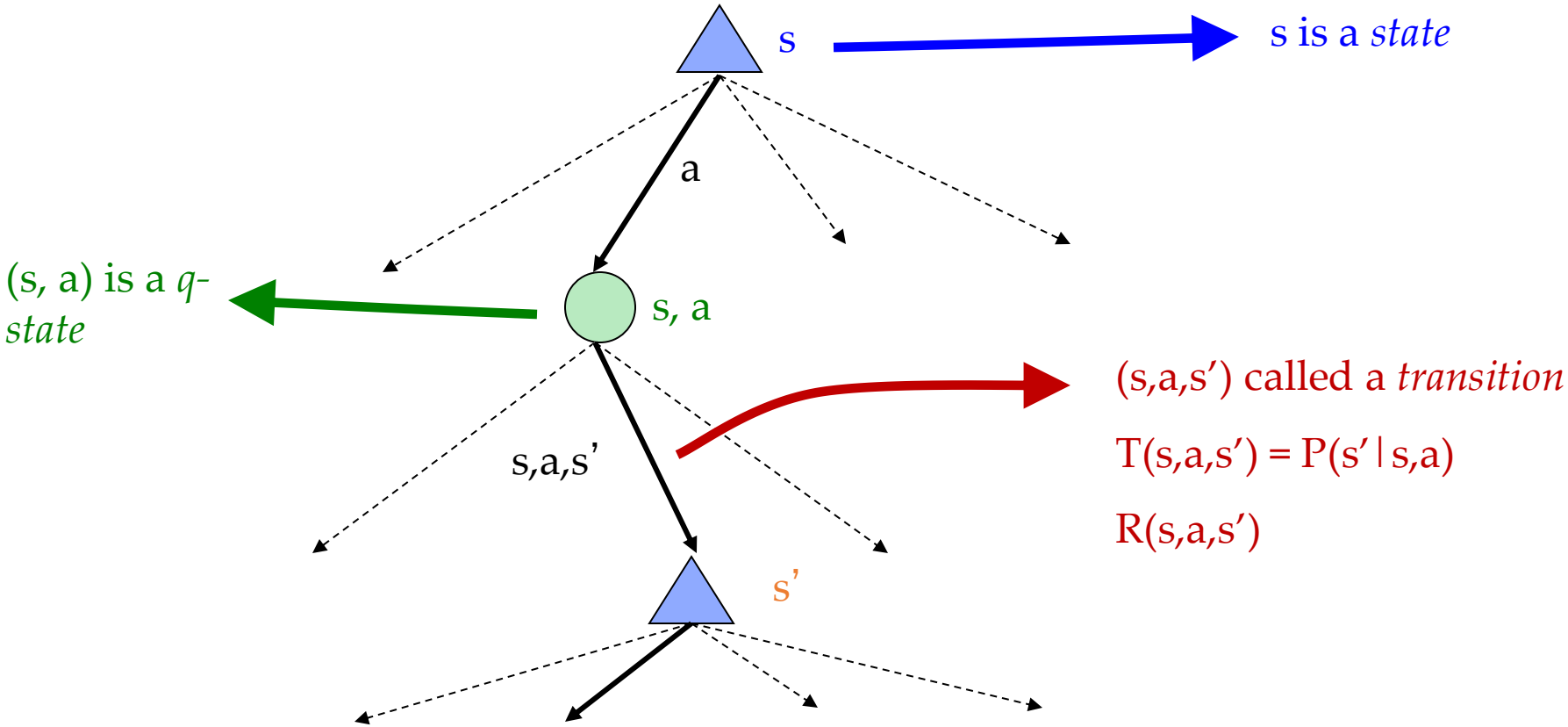


Racing Search Tree



MDP Search Trees

- Each MDP state projects an expectimax-like search tree



Utilities of Sequences

- What preferences should an agent have over reward sequences?
- More or less? [1, 2, 2] or [2, 3, 4]
- Now or later? [0, 0, 1] or [1, 0, 0]

Discounting

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- One solution: values of rewards decay exponentially

1

Worth Now

γ

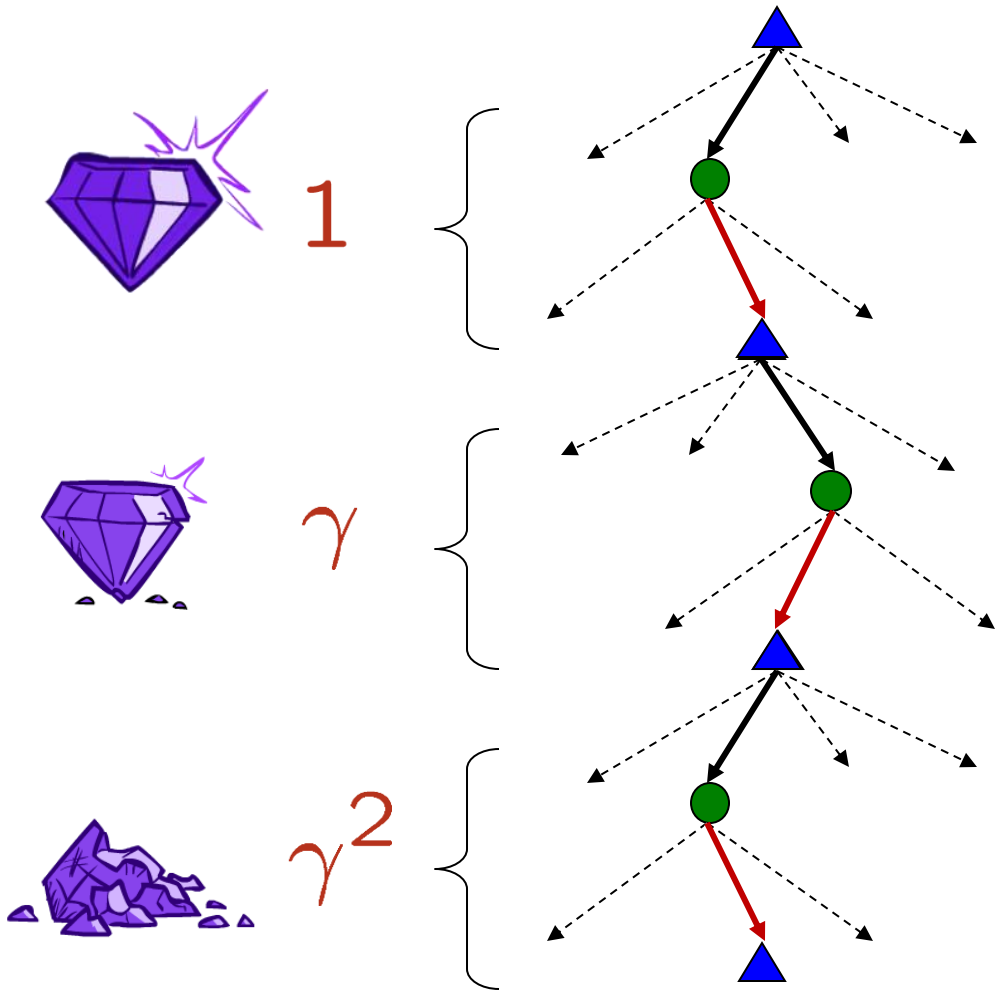
Worth Next Step

γ^2

Worth In Two Steps

Discounting

- Why discount?
  - Reward now is better than later
  - Also helps our algorithms converge
- How to discount?
  - Each time we descend a level, we multiply in the discount once
- Example: discount of 0.5
  - U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3
  - U([1,2,3]) < U([3,2,1])



Infinite Utilities?!

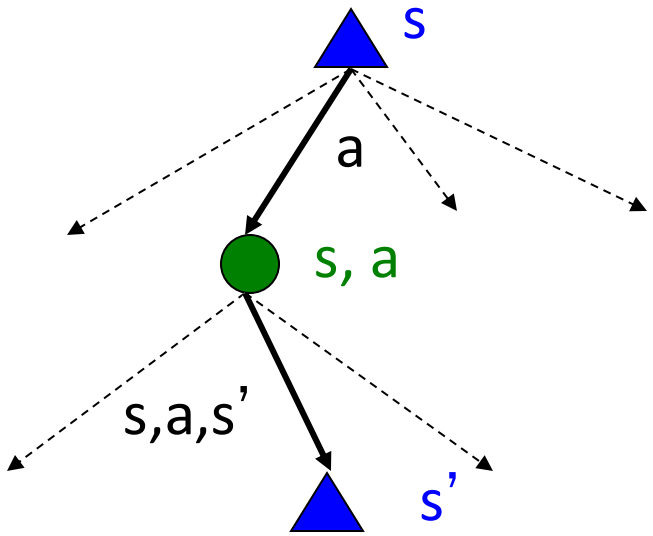
- Problem: What if the game lasts forever? Do we get infinite rewards?
- Solutions:
  - Discounting: use $0 < \gamma < 1$

$$U([r_0, \ldots r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\text{max}}/(1-\gamma)$$

    - Smaller γ means smaller "horizon" – shorter term focus
  - Finite horizon: (similar to depth-limited search)
    - Terminate episodes after a fixed T steps (e.g. life)
    - Gives nonstationary policies (π depends on time left)
  - Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like "overheated" for racing)

Recap

- Markov decision processes:
  - Set of states S
  - Start state s_0
  - Set of actions A
  - Transitions P(s'|s,a) (or T(s,a,s'))
  - Rewards R(s,a,s') (and discount γ)



- MDP quantities so far:
  - Policy = Choice of action for each state
  - Utility = sum of (discounted) rewards

Exercise

- The learner and the decision maker is the
  - A. Agent
  - B. Reward
  - C. State
  - D. Environment

Exercise

- At each step the agent takes an
  - A. Environment
  - B. State
  - C. Action
  - D. Reward

Exercise

- What does the policy function do?
  - A. infer the current state
  - B. infer the optimal action
  - C. infer the next state
  - D. compute the reward

Exercise

- What does the transition function do?
  - A. infer the current state
  - B. infer the optimal action
  - C. infer the next state
  - D. compute the reward

Exercise

- What does the reward function do?
  - A. infer the current state
  - B. infer the optimal action
  - C. infer the next state
  - D. compute the reward

Exercise

- If the reward is always +1, what is the sum of the discounted infinite reward when gamma < 1?

Exercise

- What is the difference between a small gamma (discounted factor) and a larger gamma?
  - With a smaller gamma the agent is more far-sighted and considers reward farther into the future
  - With a larger gamma the agent is more far-sighted and considers reward farther into the future
  - The size of the discounted factor has no effect on the agent

Exercise

- Support gamma=0.8 and we observe the following sequence of rewards: R1= -3, R2=5, R3=2, with T=3, What is the total utility?

Homework

- Revisit the multiplication rule of probability on your textbook
  - Joint distribution
  - Conditional distribution
  - Independence
  - …